kasta

SMART CONTRACT AUDIT

ZOKYO.

Dec 22nd, 2021 | v. 1.0

PASS

Zokyo Security team has concluded that these smart contracts pass security qualifications and are fully production-ready

TECHNICAL SUMMARY

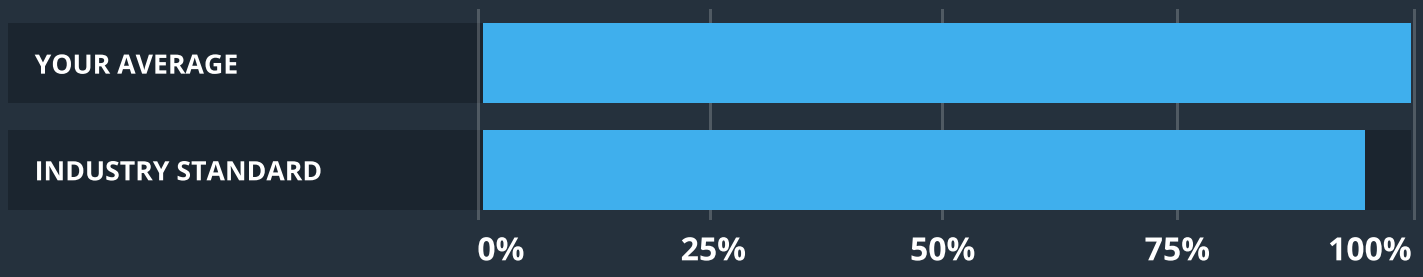
This document outlines the overall security of the Kasta smart contracts, evaluated by Zokyo's Blockchain Security team.

The scope of this audit was to analyze and document the Kasta smart contract codebase for quality, security, and correctness.

Contract Status

There was 1 critical issue found during the audit.

Testable Code



The testable code is 100%, which is above the industry standard of 95%.

It should be noted that this audit is not an endorsement of the reliability or effectiveness of the contract, rather limited to an assessment of the logic and implementation. In order to ensure a secure contract that's able to withstand the Ethereum network's fast-paced and rapidly changing environment, we at Zokyo recommend that the Kasta team put in place a bug bounty program to encourage further and active analysis of the smart contract.

TABLE OF CONTENTS

Auditing Strategy and Techniques Applied 3
Summary 5
Structure and Organization of Document 6
Complete Analysis 7
Code Coverage and Test Results for all files 10
Tests written by Zokyo Secured team 10

AUDITING STRATEGY AND TECHNIQUES APPLIED

The Smart contract's source code was taken from the Kasta repository.

Repository:
https://github.com/kasta-io/token-vesting/commit/eb41396aed28fc301f05fbbb8eb033e53e8e0b5a

Last commit:
1bb2b9b4c9674217c2f9d16a337b5b24a3ae0977

Contracts under the scope:

- KastaVesting;
- Migrations.

Throughout the review process, care was taken to ensure that the token contract:

- Implements and adheres to existing Token standards appropriately and effectively;
- Documentation and code comments match logic and behavior;
- Distributes tokens in a manner that matches calculations;
- Follows best practices in efficient use of gas, without unnecessary waste;
- Uses methods safe from reentrance attacks;
- Is not affected by the latest vulnerabilities;
- Whether the code meets best practices in code readability, etc.

Zokyo's Security Team has followed best practices and industry-standard techniques to verify the implementation of Kasta smart contracts. To do so, the code is reviewed line-by-line by our smart contract developers, documenting any issues as they are discovered. Part of this work includes writing a unit test suite using the Truffle testing framework. In summary, our strategies consist largely of manual collaboration between multiple team members at each stage of the review:

1	Due diligence in assessing the overall code quality of the codebase.
2	Cross-comparison with other, similar smart contracts by industry leaders.
3	Testing contract logic against common and uncommon attack vectors.
4	Thorough, manual review of the codebase, line-by-line.

SUMMARY

Zokyo auditing team has run a deep investigation of the Kasta smart contracts. During the auditing process, there were found some issues. The contracts are well written and structured.

There was 1 critical issue found during the audit. Were found just a couple of informational issues and 1 issue with a high severity level. All of the issues were successfully resolved by the Kasta team.

Based on the conducted audit, we give a score of 99 to this contract.

STRUCTURE AND ORGANIZATION OF DOCUMENT

For ease of navigation, sections are arranged from most critical to least critical. Issues are tagged “Resolved” or “Unresolved” depending on whether they have been fixed or addressed. Furthermore, the severity of each issue is written as assessed by the risk of exploitation or other unexpected or otherwise unsafe behavior:

Critical

The issue affects the ability of the contract to compile or operate in a significant way.

High

The issue affects the ability of the contract to compile or operate in a significant way.

Medium

The issue affects the ability of the contract to operate in a way that doesn’t significantly hinder its behavior.

Low

The issue has minimal impact on the contract’s ability to operate.

Informational

The issue has no impact on the contract’s ability to operate.

COMPLETE ANALYSIS

Administrator can withdraw all kastaTokens at any time

CRITICAL | RESOLVED

Function getBackFunds allows the administrator to withdraw all kastaTokens at any time, even if the contract has valid vestingSchedules, so users can't claim their tokens.

Recommendation:
Add restriction that doesn't allow administrator withdraw kastTokens amount more than the sum of vestingSchedule.amount.

Re-audit comment:
They have removed function getBackFunds.

Method setVestingSchedule doesn't check for vesting startTime and doesn't use the variable JAN_1_2000_SECONDS

HIGH | RESOLVED

Documentation of this method describes that startTime value may be given as a date in the future or in the past, going as far back as the year 2000. But the method doesn't check startTime value so the administrator can set time more back than the year 2000.

Recommendation:
Add requirement for check startTime value. For example:

```
require(startDay>= JAN_1_2000_SECONDS, "some msg")
```

Re-audit comment:
Fixed.

Event Debug **isn't used**

INFORMATIONAL | RESOLVED

The contract has Debug event which isn't used in the contract.

Recommendation:
Remove event Debug.

Re-audit comment:
Fixed.

Variables aren't used

INFORMATIONAL | RESOLVED

The contract has variables JAN_1_2000_DAYS and JAN_1_3000_DAYS which aren't used by the contract.

Recommendation:
Remove variables JAN_1_2000_DAYS and JAN_1_3000_DAYS.

Re-audit comment:
Fixed.

	KastaVesting	Migrations
Re-entrancy	Pass	Pass
Access Management Hierarchy	Pass	Pass
Arithmetic Over/Under Flows	Pass	Pass
Unexpected Ether	Pass	Pass
Delegatecall	Pass	Pass
Default Public Visibility	Pass	Pass
Hidden Malicious Code	Pass	Pass
Entropy Illusion (Lack of Randomness)	Pass	Pass
External Contract Referencing	Pass	Pass
Short Address/ Parameter Attack	Pass	Pass
Unchecked CALL Return Values	Pass	Pass
Race Conditions / Front Running	Pass	Pass
General Denial Of Service (DOS)	Pass	Pass
Uninitialized Storage Pointers	Pass	Pass
Floating Points and Precision	Pass	Pass
Tx.Origin Authentication	Pass	Pass
Signatures Replay	Pass	Pass
Pool Asset Security (backdoors in the underlying ERC-20)	Pass	Pass

CODE COVERAGE AND TEST RESULTS FOR ALL FILES

Tests written by Zokyo Security team

As part of our work assisting Kasta in verifying the correctness of their contract code, our team was responsible for writing integration tests using the Truffle testing framework.

Tests were based on the functionality of the code, as well as a review of the Kasta contract requirements for details about issuance amounts and how the system handles these.

Code Coverage

The resulting code coverage (i.e., the ratio of tests-to-code) is as follows:

FILE	% STMTS	% BRANCH	% FUNCS	% LINES	UNCOVERED LINES
contracts\	100.00	100.00	100.00	100.00	
KastaVesting.sol	100.00	100.00	100.00	100.00	
Migrations.sol	100.00	100.00	100.00	100.00	
All files	**100.00**	**100.00**	**100.00**	**100.00**	

Test Results

```
Contract: KastaVesting
  Methods
    constructor
      ✓ Must deploy correctly (543ms)
    grantRole
      ✓ Must fail if sender hasn't admin role (1005ms)
      ✓ Must grant the pauser role correctly (520ms)
    setVestingSchedule
      ✓ Must fail if sender hasn't admin role (222ms)
      ✓ Must fail if contract is paused (589ms)
```

```
    ✓ Must fail if start time < 01.01.2000 (243ms)
    ✓ Must fail if start time > 01.01.2030 (240ms)
    ✓ Must fail if vesting duration equals 0 (209ms)
    ✓ Must fail if vesting duration is bigger than 10 years (223ms)
    ✓ Must fail if beneficiary already has vesting schedule (455ms)
    ✓ Must set vesting schedule correctly (375ms)
  revokeSchedule
    ✓ Must fail if caller isn't admin (192ms)
    ✓ Must fail if vesting schedule isn't valid (229ms)
    ✓ Must fail if vesting schedule isn't revocable (521ms)
    ✓ Must fail if vesting schedule has already ended (481ms)
    ✓ Must fail if the contract doesn't have enough tokens (496ms)
    ✓ Must revoke correctly (724ms)
    ✓ Must revoke correctly after user's claiming (1354ms)
  claim
    ✓ Must fail if vesting schedule isn't valid (193ms)
    ✓ Must fail if vesting schedule is revoked (695ms)
    ✓ Must fail if claimable amount equals 0 (419ms)
    ✓ Must fail if the contract doesn't have enough tokens (430ms)
    ✓ Must claim all tokens after ending of vesting schedule (760ms)
    ✓ Must claim 50% of tokens in the middle of vesting schedule (754ms)
    ✓ Must claim tokens with an odd period (1150ms)
    ✓ Must claim tokens with small amount of vested tokens and max duration (1193ms)
  vestingForAccountAsOf
    ✓ Must fail if caller isn't admin or beneficiary (115ms)
    ✓ Must return correct data of vesting schedule in the middle of vesting period (339ms)
    ✓ Must return correct data of vesting schedule after ending of vesting period (287ms)
  vestingAsOf
    ✓ Must return correct data of vesting schedule in the middle of vesting period (348ms)
    ✓ Must return correct data of vesting schedule after ending of vesting period (320ms)
  pause
    ✓ Must fail if caller hasn't pauser role (179ms)
    ✓ Must pause the contract correctly (459ms)
  unpause
    ✓ Must fail if caller hasn't pauser role (214ms)
    ✓ Must unpause the contract correctly (1167ms)
```

```
Contract: Migrations
  Methods
    owner
      ✓ Must save owner's address correctly (97ms)
    setCompleted
      ✓ Must fail to update if caller isn't owner (184ms)
      ✓ Must update correctly (486ms)

38 passing (23s)
```

We are grateful to have been given the opportunity to work with the Kasta team.

The statements made in this document should not be interpreted as investment or legal advice, nor should its authors be held accountable for decisions made based on them.

Zokyo's Security Team recommends that the Kasta team put in place a bug bounty program to encourage further analysis of the smart contract by third parties.

ZOKYO.